Practical No. 6

C Program: FCFS and SJF Scheduling (Waiting Time, Turnaround Time, Gantt Chart)

1. AIM

To write a C program that calculates the waiting time and turnaround time of processes using FCFS and non-preemptive SJF CPU scheduling, displays the Gantt chart, and finds the average waiting time and average turnaround time. (All processes arrive at time 0.)

2. REQUIREMENTS

- Operating System: Ubuntu / Linux (or Windows)
- Compiler: GCC (C compiler)
- Text editor: Nano / Vim / Gedit / VS Code / Code::Blocks
- Terminal to compile and run
- Hardware: computer, keyboard (input), monitor (output)

3. SHORT THEORY

CPU scheduling: the OS picks one process from the ready queue and gives it the CPU. Good scheduling reduces waiting time and improves performance.

- FCFS (First Come First Served): processes run in the order they arrive. Non-preemptive.
- SJF (Shortest Job First): the process with the smallest burst time runs first. Here it is non-preemptive (a running process is never interrupted).
- Burst Time (BT): total CPU time a process needs.
- Waiting Time (WT): time spent waiting in the ready queue.
- Turnaround Time (TAT): total time from arrival to completion.
- Gantt Chart: a bar/line diagram showing the order of execution with start and end times.

Formula	Note
`WT = Start Time - Arrival Time`	Since arrival = 0, WT = start time.
`TAT = Completion Time - Arrival Time`	Same as WT + BT when arrival = 0.
`Average WT = Total WT / n`	n = number of processes.
`Average TAT = Total TAT / n`	n = number of processes.

4A. CODE: FCFS (file name: fcfs.c)

```
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], wt[20], tat[20];
    int total_wt = 0, total_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter burst time of P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }
```

```
    wt[0] = 0;
    for (i = 1; i < n; i++)
        wt[i] = wt[i - 1] + bt[i - 1];

    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
        total_wt += wt[i];
        total_tat += tat[i];
    }

    printf("\nGantt Chart:\n");
    printf("0");
    for (i = 0; i < n; i++)
        printf(" -- P%d -- %d", i + 1, tat[i]);

    printf("\n\nProcess\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);

    printf("\nAverage Waiting Time = %.2f", (float)total_wt / n);
    printf("\nAverage Turnaround Time = %.2f\n", (float)total_tat / n);
    return 0;
}
```

4B. CODE: SJF (file name: sjf.c)

```
#include <stdio.h>

int main() {
    int n, i, j, temp;
    int pid[20], bt[20], wt[20], tat[20];
    int total_wt = 0, total_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("Enter burst time of P%d: ", pid[i]);
        scanf("%d", &bt[i]);
    }

    /* Sort by burst time (shortest first) */
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (bt[j] > bt[j + 1]) {
                temp = bt[j];  bt[j] = bt[j + 1];   bt[j + 1] = temp;
                temp = pid[j]; pid[j] = pid[j + 1]; pid[j + 1] = temp;
            }
        }
    }

    wt[0] = 0;
    for (i = 1; i < n; i++)
        wt[i] = wt[i - 1] + bt[i - 1];

    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
        total_wt += wt[i];
        total_tat += tat[i];
    }

    printf("\nGantt Chart:\n");
    printf("0");
    for (i = 0; i < n; i++)
        printf(" -- P%d -- %d", pid[i], tat[i]);
```

```
    printf("\n\nProcess\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\n", pid[i], bt[i], wt[i], tat[i]);

    printf("\nAverage Waiting Time = %.2f", (float)total_wt / n);
    printf("\nAverage Turnaround Time = %.2f\n", (float)total_tat / n);
    return 0;
}
```

Important: the SJF code on the assignment slides has no sorting step, so it behaves exactly like FCFS. The sorting block marked 'Sort by burst time' above is added so that SJF really runs the shortest job first and matches the sample output in the assignment.

5. HOW TO EXECUTE (step by step)

Command / Key	What it does
`Ctrl + Alt + T`	Open the Ubuntu terminal.
`sudo apt install gcc`	Only if gcc is not installed. Check with: gcc --version
`cd ~`	Go to the home folder.
`nano fcfs.c`	Create the FCFS file, paste the code, save (Ctrl+O, Enter, Ctrl+X).
`gcc fcfs.c -o fcfs`	Compile. gcc is the compiler, -o fcfs names the output program fcfs.
`./fcfs`	Run the program. Enter number of processes and burst times.
`nano sjf.c`	Create the SJF file, paste the code, save and exit.
`gcc sjf.c -o sjf`	Compile the SJF program.
`./sjf`	Run it with the same input and compare the results.

6. LINE-BY-LINE EXPLANATION (FCFS)

Code	Meaning
`#include <stdio.h>`	Adds the standard input/output library, needed for printf and scanf.
`int main() {`	Program starts here. { opens the main function.
`int n, i;`	n = number of processes, i = loop counter.
`int bt[20], wt[20], tat[20];`	Arrays for burst time, waiting time and turnaround time (up to 20 processes).
`int total_wt = 0, total_tat = 0;`	Variables to add up all waiting and turnaround times. Start at 0.
`printf("Enter number of processes: ");`	Asks the user for the number of processes.
`scanf("%d", &n);`	Reads an integer and stores it in n.
`for (i = 0; i < n; i++)`	Loop that repeats once for every process.
`scanf("%d", &bt[i]);`	Reads the burst time of each process into array bt.
`wt[0] = 0;`	The first process does not wait, so its waiting time is 0.
`wt[i] = wt[i-1] + bt[i-1];`	Waiting time of a process = waiting time of previous + burst time of previous.
`tat[i] = wt[i] + bt[i];`	Turnaround time = waiting time + burst time.

Code	Meaning
`total_wt += wt[i];`	Adds this waiting time to the total. (+= means add and store.)
`total_tat += tat[i];`	Adds this turnaround time to the total.
`printf("0"); ... " -- P%d -- %d"`	Prints the Gantt chart. Starts with 0, then each process name and its end time. End time = TAT because arrival is 0.
`printf("\nProcess\tBT\tWT\tTAT\n");`	Prints the table heading. \t = tab space, \n = new line.
`printf("P%d\t%d\t%d\t%d\n", ...);`	Prints one row per process: name, BT, WT, TAT.
`(float)total_wt / n`	(float) converts to decimal so the average is not cut off to an integer.
`printf("... %.2f", ...);`	%.2f prints a decimal number with 2 digits after the point.
`return 0;`	Ends the program normally.

7. EXTRA LINES IN SJF

Code	Meaning
`int pid[20];`	Stores the process number (1, 2, 3...), so we still know which process is which after sorting.
`pid[i] = i + 1;`	Gives process IDs 1, 2, 3, ...
`for (i...) for (j...)`	Bubble sort: two loops that compare neighbouring burst times.
`if (bt[j] > bt[j+1])`	If the left burst time is bigger, they are in the wrong order, so swap them.
`temp = bt[j]; bt[j] = bt[j+1]; bt[j+1] = temp;`	Swaps two burst times using a temporary variable.
`(same swap for pid)`	Swaps the process IDs too, so each process stays with its own burst time.
`(rest of the code)`	Same as FCFS, but now the array is sorted, so the shortest job runs first.

8. MEANING OF SYMBOLS

Symbol	Use
`%d`	Format code for an integer in printf / scanf.
`%.2f`	Format code for a decimal number with 2 decimal places.
`&`	Address-of. scanf needs &n so it can store the value inside n.
`\n`	New line.
`\t`	Tab space. Used to line up table columns.
`[ ]`	Array index. bt[0] is the burst time of the first process.
`+=`	Add and store: total += x means total = total + x.
`i++`	Increase i by 1.
`(float)`	Type cast. Converts an int to decimal.
`{ }`	Start and end of a block of code.

Symbol	Use
`;`	Ends a statement in C.

9. SAMPLE INPUT AND OUTPUT

Input: 4 processes with burst times P1 = 5, P2 = 3, P3 = 8, P4 = 6 (all arrive at 0).

Gantt chart	Calculation
`0 | P1 | 5 | P2 | 8 | P3 | 16 | P4 | 22`	WT: P1=0, P2=5, P3=8, P4=16. TAT: 5, 8, 16, 22. Avg WT = 29/4 = 7.25. Avg TAT = 51/4 = 12.75
`0 | P2 | 3 | P1 | 8 | P4 | 14 | P3 | 22`	WT: P2=0, P1=3, P4=8, P3=14. TAT: 3, 8, 14, 22. Avg WT = 25/4 = 6.25. Avg TAT = 47/4 = 11.75

FCFS gives Avg WT 7.25, SJF gives Avg WT 6.25. SJF is better (lower waiting time).

10. CONCLUSION

Thus, we have implemented FCFS and non-preemptive SJF scheduling in C, displayed their Gantt charts and calculated waiting time, turnaround time and their averages. For the sample input, SJF gave a lower average waiting time (6.25) than FCFS (7.25), so SJF performs better.

11. QUICK VIVA ANSWERS

- CPU scheduling: choosing which process in the ready queue gets the CPU next. Required to use the CPU efficiently and keep the system responsive.
- FCFS: the process that comes first runs first, like a queue. E.g. P1=5, P2=3, P3=8 run in that order; waiting times are 0, 5, 8.
- Non-preemptive SJF: the shortest burst time runs first and is not interrupted. E.g. P1=5, P2=3, P3=8 run as P2, P1, P3.
- WT vs TAT: waiting time is time spent waiting in the queue; turnaround time is total time from arrival to completion (TAT = WT + BT).
- Gantt chart: a timeline showing which process runs when. It makes it easy to calculate WT and TAT.
- Lower average WT: SJF, because short jobs go first so fewer processes wait behind long ones.
- Convoy effect: in FCFS, many short processes get stuck waiting behind one long process.
- SJF advantage: minimum average waiting time. Disadvantage: long processes may starve, and burst time must be known in advance.